

Sample in place

Document number: P0469R0

Date: 2016-10-17

Audience: Library Evolution Working Group

Reply-to: R. "Tim" Song <rs2740@gmail.com>

Abstract

This paper proposes adding an algorithm `inplace_sample` that is like `std::sample`, but partitions the input range instead of copying the sampled elements.

Background

The [random generator proposal](#), based on Melissa O'Neill's `randutils` library, included a member function named `sample` that performed sampling in-place. According to Walter Brown, there is interest in a standalone `inplace_sample` algorithm, hence this proposal.

Motivation

There are two drawbacks for `std::sample`:

- It copies the sampled elements, which could be potentially expensive, and makes it hard to use with move-only types.
- There is no easy way to get the elements *not* included in the sample.

`inplace_sample` solves both problems. Instead of copying elements, it partitions them so that every element in the sample comes before every element not included in the sample. And since it returns the partition point, accessing the remaining elements is trivial.

Impact on the standard

This is a pure extension.

Design questions

Stability

The stability guarantee in the specification comes from the original implementation's use of `stable_partition`, but it does impose a cost in the form of additional memory or move/swap operations (as opposed to the case in the selection sampling version of `std::sample`, which gets it for free).

It may be appropriate to remove the requirement.

Naming

There appears to be two naming conventions in `<algorithm>` for non-copying and copying variants of the same algorithm:

- One appends `_copy` to the copying variant, hence `partition/partition_copy` and `remove/remove_copy`, etc.
- The other prepends `inplace_` to the non-copying variant: `merge/inplace_merge`

The second alternative is chosen in this paper since it seems too late to rename `std::sample`.

Proposed specification

Add the following signature to `[algorithms.general]`, header `<algorithm>` synopsis:

```
template <class ForwardIterator, class Distance,
          class UniformRandomBitGenerator>
ForwardIterator inplace_sample(ForwardIterator first, ForwardIterator last,
                              Distance n, UniformRandomBitGenerator&& g);
```

Add to the end of [alg.random.sample]:

```
template <class ForwardIterator, class Distance,
          class UniformRandomBitGenerator>
ForwardIterator inplace_sample(ForwardIterator first, ForwardIterator last,
                              Distance n, UniformRandomBitGenerator&& g);
```

-?- *Requires:*

- ForwardIterator shall satisfy the requirements of ValueSwappable ([swappable.requirements]).
- The type of *first shall satisfy the requirements of MoveConstructible (Table [tab:moveconstructible]) and MoveAssignable (Table [tab:moveassignable]).
- Distance shall be an integer type.
- remove_reference_t<UniformRandomBitGenerator> shall meet the requirements of a uniform random bit generator type ([rand.req.urng]) whose return type is convertible to Distance.

-?- *Effects:* If $n \geq \text{last} - \text{first}$, has no effects. Otherwise, reorders elements in the range [first, last), such that each possible sample of size n has equal probability of appearing in the range [first, first + n). The relative order of the elements both inside the sample and outside the sample is preserved.

-?- *Returns:* first + min(n , last - first).

-?- *Remarks:* To the extent that the implementation of this function makes use of random numbers, the object g shall serve as the implementation's source of randomness.

Possible implementation

The following naïve implementation assumes a version of `std::stable_partition` that accepts forward iterators.

```
template <class ForwardIterator, class Distance,
          class UniformRandomBitGenerator>
ForwardIterator inplace_sample(ForwardIterator first, ForwardIterator last,
                              Distance n, UniformRandomBitGenerator&& g);
{
    auto total = std::distance(first, last);
    using dist_t = uniform_int_distribution<Distance>;
    using param_t = typename dist_t::param_type;
    dist_t d{};

    return std::stable_partition(first, last,
        [&](const auto&) {
            --total;
            if (d(g, param_t{0, total}) < n) {
                --n;
                return true;
            } else {
                return false;
            }
        });
}
```